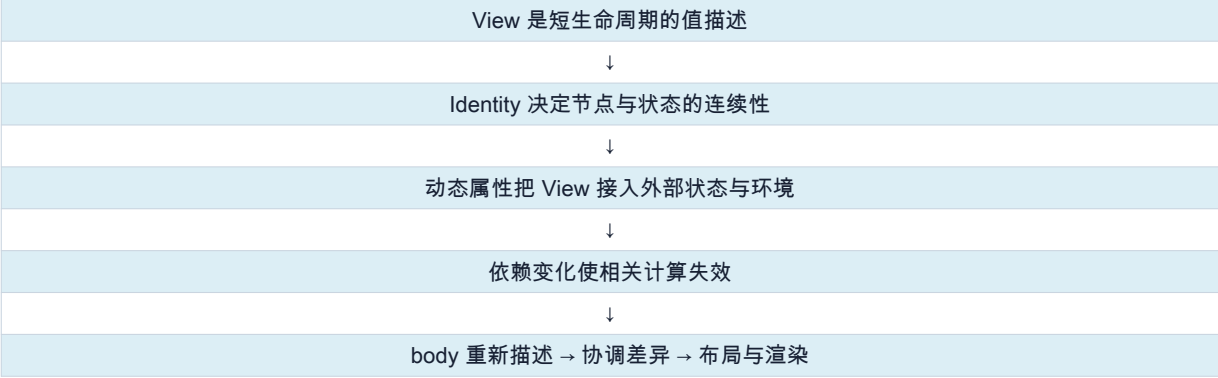


SWIFTUI 深度讲义

从状态包装器到渲染流水线

@State · Binding · ObservableObject · Environment · Identity · Observation · Diff & Render

本讲义的主线



面向没有对话背景的读者 · 适合 A4 双面打印

修订日期：2026-07-28

如何阅读这份讲义

这不是一份 API 速查表，而是一条逐步建立心智模型的学习路径。读者不需要预先理解 SwiftUI 的内部实现；只要熟悉 Swift 的 struct、class、闭包和基础 SwiftUI 语法，就可以从第一章开始。

最重要的阅读规则 每当看到“内部可能类似”“概念上可以理解为”时，都应把它当作解释公开行为的模型，而不是 Apple 已公开的源码。SwiftUI 的 AttributeGraph、状态位置与渲染协调细节并未完整公开。

如何阅读“教学伪代码”

本讲义会在仅靠 API 声明仍难以形成直觉的地方，加入统一标识的教学伪代码。每一段都会写清楚三个信息：

- 推导依据：来自公开 API、Swift 语言规则、当前 SDK interface，或可以重复观察到的框架行为。
- 解释目的：这段模型专门回答哪个问题，例如 State 如何复用、Binding 如何转发写入。
- 边界声明：伪代码不代表 Apple 的真实类型布局、函数名、调度顺序或源码。

伪代码里的 ``runtime``、``node``、``location``、``scheduler`` 等名称只是教学符号。应使用它理解“责任如何分工”和“数据如何流动”，不能据此依赖私有实现，也不能把某一行的先后顺序当作框架契约。

本书解决的核心问题

- `@State` 展开后是什么？为什么 struct View 能在非 mutating 上下文中修改它？
- `nonmutating set` 到底限制了什么？写 `self.storage.value = ...` 为什么仍然可能合法？
- View value 每次都可能重新生成，状态为何仍能保留？Identity 从哪里来？
- `DynamicProperty.update()` 在 body 前做什么？为什么不是重复“读取新值”？
- `@Binding` 如何连接正确的 State storage？它是否保存父 View 的 Identity？
- `@StateObject` 与 `@ObservedObject` 的差别为何首先是所有权，而不是语法？
- `@State` 与 `@StateObject` 的初始化为何一个接收值、一个延迟执行对象创建表达式？
- Environment 在 View Tree 中如何保存、继承、覆盖和查找？
- `@EnvironmentObject` 为什么既像 Environment，又像 ObservedObject？
- SwiftUI 如何建立依赖、选择失效入口，并避免把整个 App 都重算一遍？
- `ObservableObject` 与 Observation 的更新粒度有何本质差异？
- `body` 重算、子 View 的 body 求值、布局和真实渲染为何必须分开理解？

术语边界

术语	在本讲义中的含义
View value	由 Swift struct 表示的短生命周期界面描述值。
View node	SwiftUI 运行时在视图层级中维护的逻辑节点；不是公开类型。
Identity	SwiftUI 跨更新匹配逻辑节点的依据，包含结构身份与显式身份。
State storage	由 SwiftUI 管理、与视图身份生命周期关联的状态存储；具体私有结构不作断言。
失效 invalidation	某个计算节点被标记为需要重新求值，不等同于像素已经重绘。
Render	将已协调、布局后的变化提交给底层显示系统；不简单等同于创建 UIView。
教学伪代码	根据公开接口与可观察行为构建的解释模型；不是 Apple 源码。

目录与学习路线

第一部分 Swift 语言基础：Property Wrapper 与 nonmutating set

第二部分 SwiftUI 的持久性来源：View、Identity 与 State 生命周期

第三部分 动态输入：DynamicProperty 与 View 工作副本

第四部分 状态所有权：Binding、ObservedObject、StateObject 的展开与实现模型

第五部分 树状上下文：Environment 与 EnvironmentObject

第六部分 依赖与更新：Dependency Tracking、AttributeGraph、Observation

第七部分 完整流水线：从用户操作到屏幕像素

第八部分 状态工具比较、常见误区与调试方法

附录 术语速查、实验清单与官方资料

第一部分 从 Swift 语言机制理解 @State

1. `@State` 首先是一个 Property Wrapper

看到 `@State private var count = 0` 时，不要先把它理解成“带自动刷新的变量”。第一步应该回到 Swift 的 Property Wrapper 语法：编译器把声明分成包装器存储、包装值访问和投影值访问。

我们写下的代码

```
struct CounterView: View {
    @State private var count = 0

    var body: some View {
        Button("count = \(count)") {
            count += 1
        }
    }
}
```

概念上，编译器会生成类似下面的结构。真实生成代码受 Swift 版本和访问控制影响，但三个名字之间的关系是稳定的：

用于理解的近似展开

```
private var _count: State<Int> = State(wrappedValue: 0)

private var count: Int {
    get { _count.wrappedValue }
    nonmutating set { _count.wrappedValue = newValue }
}

// $count 来自 _count.projectedValue，类型是 Binding<Int>
```

写法	实际角色	典型类型
count	wrapped value：业务代码直接读写的值	Int
_count	wrapper storage：编译器生成的包装器属性	State<Int>
\$count	projected value：包装器公开的投影	Binding<Int>

先记住 `@State` 同时利用了两套机制：Swift 编译器负责 Property Wrapper 语法；SwiftUI 运行时负责状态存储、生命周期和更新。只理解其中一套都不完整。

工具链版本说明 本讲义以长期存在的 State property-wrapper 语义建立模型。本次校验所用 Xcode 26.5 SDK 仍公开 `struct State<Value>`；Apple 的较新文档还提示 Xcode 27 及以后可能采用 `State()` 宏。宏或 wrapper 的生成形式可以演进，但“SwiftUI 管理存储、Identity 决定生命周期、投影产生 Binding”仍是应掌握的行为契约。

2. 普通 setter 为什么默认会修改整个 struct

```
struct Counter {
    var value = 0
}

let counter = Counter()
counter.value = 1
// error: Cannot assign to property: 'counter' is a 'let' constant
```

值类型的存储属性属于值本身。给 `value` 赋值意味着产生一个内容不同的 `Counter` 值，因此 setter 默认具有 `mutating` 语义；在不可变的 `self` 上不能调用这种 setter。

```
struct Wrapper<Value> {
    private var value: Value

    var wrappedValue: Value {
        get { value }
        set { value = newValue } // 默认 mutating
    }
}
```

3. `nonmutating set` 的准确含义

准确翻译 `nonmutating set` 不是“setter 里不能写 self”，也不是“省略 self 就不算修改”。它表示：调用 setter 不需要把这个值类型的 self 当成可变值。

在 `nonmutating` setter 中，可以读取 `self`，也可以通过它持有的引用修改引用对象；但不能替换 `self` 的存储属性，因为后者会改变 wrapper struct 自身的值。

可运行的简化模型

```
final class Box<Value> {
    var value: Value
    init(_ value: Value) { self.value = value }
}

@propertyWrapper
struct MyState<Value> {
    private let box: Box<Value>

    init(wrappedValue: Value) {
        box = Box(wrappedValue)
    }

    var wrappedValue: Value {
        get { box.value }
        nonmutating set {
            self.box.value = newValue // 合法：修改 Box 对象
        }
    }
}
```

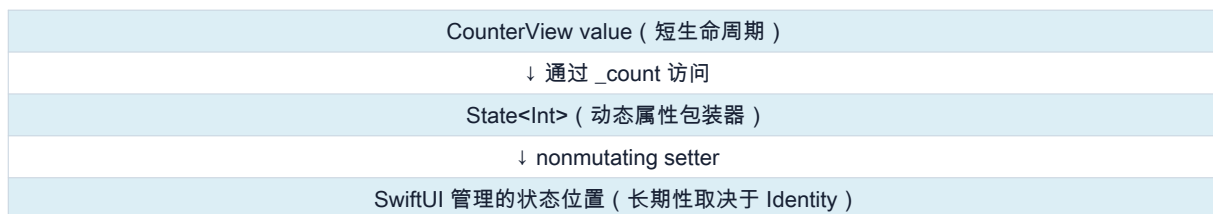
setter 内的写法	是否合法	原因
self.box.value = newValue	是	修改引用对象的内容，wrapper 的引用字段没变。
box.value = newValue	是	与上一行语义相同，只是省略 self。
self.box = Box(newValue)	否	试图替换 wrapper 自己的存储属性。
self.localValue = newValue	否	直接改变值类型 self 的存储。

关键区别 `self.box.value = ...` 与 `self.box = ...` 看起来只差 `.value`，但前者改变堆上的对象，后者改变 struct 内保存的引用值。

4. 为什么 struct View 的按钮闭包能修改 `@State`

SwiftUI 的 View 通常是 struct，`body` 的 getter 也不是 mutating。按钮动作捕获的 `self` 不能按普通存储属性的方式修改。但是 `State.wrappedValue` 提供 nonmutating setter，因此 `count += 1` 不要求把当前 View value 变成可变量。

概念关系



上图中的“状态位置”是心智模型。公开 API 能确认 SwiftUI 管理 State 的存储和生命周期，但不能据此断言 `State` 内部永远只是一根简单指针；真实实现可能包含初值、位置对象、缓存或图属性等更多信息。

教学伪代码 | State 从初值安装到身份存储 推导依据：Apple 文档说明 State 的存储由 SwiftUI 管理；同一 Identity 下状态保留，身份结束后重新采用初值。目的：解释临时 State wrapper 为什么能在多次 View value 重建之间访问同一个值。边界：只解释公开行为，不代表 Apple 的类型布局、函数名、调度顺序或真实源码。

教学伪代码 (非 Apple 源码)

```
func installState<Value>(
  wrapper: inout State<Value>,
  node: ViewNode,
  field: FieldID
){
  let location = node.stateLocations[field]
  ?? StateLocation(initial: wrapper.initialValue)

  node.stateLocations[field] = location
  wrapper.location = location
}

func writeState<Value>(
  _ location: StateLocation<Value>,
  _ newValue: Value
){
  location.value = newValue
  invalidateReaders(of: location)
}
```

4.1 State 保存 class 引用时，引用变化和对象内部变化不同

```
final class LegacyModel: ObservableObject {
  @Published var count = 0
}

struct Demo: View {
  @State private var model = LegacyModel()

  var body: some View {
    Button("\(model.count)") {
      model.count += 1
    }
  }
}
```

在经典 ObservableObject 体系中，State 只管理 `LegacyModel` 这个引用值；给 `model` 换成另一个实例能改变 State，但修改同一实例的 Published 属性不会让 State 自动订阅 objectWillChange。需要同时观察对象内部变化时，旧体系使用 StateObject。

若 class 使用 iOS 17 的 `@Observable` 宏，属性 getter/setter 会通过 Observation 建立依赖，因此 `@State + @Observable` 才能同时表达“View 身份持有引用”和“按属性观察内部变化”。不要把两代机制混用后期待相同行为。

第二部分 View、Identity 与 State 生命周期

5. View value 不是屏幕上的长期对象

UIKit/AppKit 中的 `UIView`/`NSView` 是 class：对象地址天然可以作为身份，也能在对象里保存长期状态。SwiftUI 的 `View` 通常是轻量值。每次父 View 的 body 重新求值，都可能构造新的子 View value。

```
let first = TitleView(title: "Hello")
let second = TitleView(title: "World")

// first 与 second 是两个不同的 struct value；
// 但 SwiftUI 可以把它们匹配为同一个逻辑 View 节点。
```

最重要的区分“View value 重新创建”不等于“逻辑 View 身份改变”，也不等于“底层显示对象被销毁重建”。

层次	会发生什么	决定因素
View value	body 求值时反复生成	Swift 代码与当前输入
逻辑节点	在更新之间被匹配或替换	Identity
状态存储	复用、创建或释放	节点 Identity 的生命周期
底层呈现	必要时更新布局/绘制	协调后的差异

6. Identity 回答“它是不是原来的那个”

Apple 将 SwiftUI 的核心概念归纳为 identity、lifetime 与 dependencies。Identity 不是用户通常能读取到的 UUID，而是框架用来跨更新识别逻辑元素的规则。

6.1 结构身份 (structural identity)

当没有显式 ID 时，SwiftUI 可以利用 View 的静态泛型结构、分支位置和容器中的结构位置形成隐式身份。更准确地说，它不只是“数组下标”；ViewBuilder 产生的泛型结构也是身份的一部分。

```
VStack {
  Text("A")
  CounterView()
}

// 概念路径：
// VStack
// └─ child 0: Text
//    └─ child 1: CounterView
```

如果下一次仍在同一个结构位置得到同一种节点，SwiftUI 能将新 value 与旧逻辑节点匹配，继续使用该节点关联的状态。普通输入值变化，例如 `TitleView(title: "Hello")` 变为 `TitleView(title: "World")`，通常不会单独改变结构身份。

6.2 显式身份 (explicit identity)

显式身份来自稳定的业务 ID，例如 `ForEach(items)` 中元素的 `Identifiable.id`，或 `.id(value)`。它为动态集合或明确的身份切换提供匹配依据。

```
ForEach(users) { user in
  UserRow(user: user) // user.id 是显式身份
}

DetailView(user: user)
.id(user.id) // user.id 变化表示一个新节点
```

教学伪代码 | 结构身份与显式身份的组合 推导依据：WWDC21 将 Identity 区分为结构身份和显式身份；ForEach 与 `.id()` 的公开行为证明稳定 ID 参与新旧节点匹配。目的：给后面的 State 生命周期和 diff 提供一个统一的 identity key 心智模型。边界：真实 Identity 不一定是可打印路径，也不保证只由这三个字段组成。

教学伪代码（非 Apple 源码）

```
func identity(
    parent: Identity,
    structuralSlot: Int,
    viewType: Any.Type,
    explicitID: AnyHashable?
) -> Identity {
    if let explicitID {
        return parent.child(
            explicit: explicitID,
            type: viewType
        )
    }

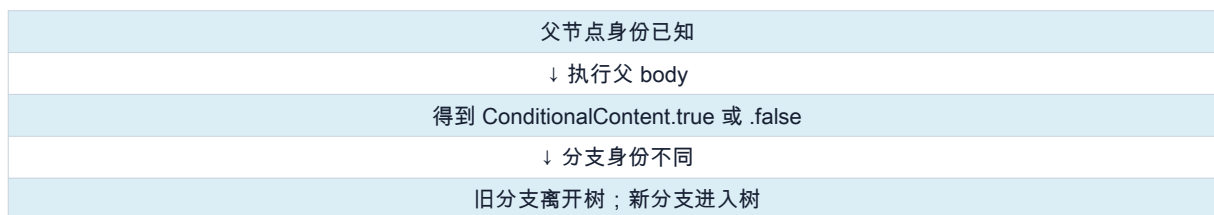
    return parent.child(
        structuralSlot: structuralSlot,
        type: viewType
    )
}
```

7. `if`、`switch` 与 `\_ConditionalContent`

ViewBuilder 会把条件分支编码进返回类型。概念上，`if/else` 形成类似 `\_ConditionalContent<TrueContent, FalseContent>` 的结构；true 分支和 false 分支代表不同身份。

```
if isLoggedIn {
    HomeView()
} else {
    LoginView()
}
```

分支切换



因此从 `LoginView` 切换到 `HomeView` 不是“同一对象改了外观”，而是一个分支节点被移除、另一个分支节点被创建。旧分支内部的 `@State`、焦点、任务和局部状态通常随身份生命周期结束。

设计提示 如果只是同一内容的颜色、位置或修饰符变化，尽量保持同一个 View 结构并改变参数；如果语义上确实是不同界面，则使用分支，接受不同身份带来的生命周期。

8. `TupleView` / ViewBuilder 与多个子 View

早期 SwiftUI 常用 `TupleView` 描述 ViewBuilder 中的多个静态子项；现代实现细节会演进，但关键思想不变：编译器把看似多行的声明组合为一个静态、类型化的内容结构。SwiftUI 因此能知道“第一个位置是什么类型、第二个位置是什么类型”。

```
VStack {
  Text("A")
  Text("B")
  CounterView()
}

// 不是运行时返回 [AnyView] ;
// 而是由 ViewBuilder 形成静态组合类型。
```

这也是随意擦除类型或改变结构位置可能影响身份、优化与动画的原因之一。

9. `ForEach`：动态集合必须用稳定业务 ID

静态 ViewBuilder 的子项数量和类型在编译期结构里相对稳定；动态集合则需要显式身份来匹配元素。

```
struct Item: Identifiable {
  let id: UUID // 创建时确定，之后保持稳定
  var title: String
}

ForEach(items) { item in
  ItemRow(item: item)
}
```

假设列表从 `[A, B, C]` 删除 A。若使用稳定 ID，B 仍匹配 B，C 仍匹配 C；若错误地把会变化的位置当身份，状态可能跟着位置而不是业务实体移动。

做法	插入/删除后	风险
稳定业务 ID	按实体匹配	最低
items.indices	索引随集合变化	行状态错位、动画异常
计算属性每次 UUID()	每次读取都变成新身份	所有行反复重建
.id(UUID())	每次 body 都强制换身份	状态、焦点、任务持续重置

教学伪代码 | ForEach 按稳定 ID 匹配元素 推导依据：ForEach 要求数据可识别；插入、删除和移动时，SwiftUI 会让同一 ID 的元素保持逻辑连续性。目的：解释为什么删除 A 后，B 的局部 State 应继续属于 B，而不是转交给新的数组下标。边界：真实集合协调还处理顺序、移动、事务、动画和惰性容器，不是一个普通 Dictionary.map。

教学伪代码（非 Apple 源码）

```
func reconcileItems<Item: Identifiable>(
  oldNodes: [Item.ID: ViewNode],
  newItems: [Item]
) -> [ViewNode] {
  newItems.map { item in
    if let old = oldNodes[item.id] {
      return reuse(old, with: item)
    } else {
      return createNode(for: item)
    }
  }
  // oldNodes 中未被新 ID 命中的节点结束生命周期
}
```

10. State 的初始值：何时采用，何时忽略

声明 `@State private var count = 0` 时，`0` 会成为包装器提供给 SwiftUI 的初始值。关键规则是：初始值用于创建新的状态存储；当同一身份已有存储时，后续新 View value 携带的初值不会覆盖现有状态。

State 初值选择

新 View value 携带 State(initialValue: 0)
↓ 根据当前节点 Identity 查找状态位置
已存在 → 继续旧值 (例如 10)
不存在 → 创建新位置并采用 0

```
struct DetailView: View {
    let incomingName: String
    @State private var draftName: String

    init(incomingName: String) {
        self.incomingName = incomingName
        _draftName = State(initialValue: incomingName)
    }
}
```

若父级从 `Tom` 改为 `Jack`，而 `DetailView` 的 Identity 不变，`draftName` 仍可能保持用户已经编辑过的值。这不是 bug，而是 `@State` 的所有权语义：它是本地状态，不是外部输入的镜像。

判断问题 如果它是“编辑草稿”，只初始化一次很合理；如果它必须始终等于父级输入，就不应复制进 State，而应直接读取值或使用 Binding。

11. Identity 改变会影响的不只是 `@State`

- `@State` 存储重新建立，采用新的初值。
- `@StateObject` 所持有的对象随旧身份结束，并为新身份创建新对象。
- 与节点绑定的 `.task` 可能被取消，新节点出现时重新启动。
- 焦点、手势、滚动位置、动画连续性等局部运行时状态可能失去。
- `onDisappear` / `onAppear` 可能依次发生，但它们不是识别 identity 的精确 API。

12. body 之前，当前 View 的 Identity 从哪里来

这是一个常见的“先有鸡还是先有蛋”疑问：如果 body 尚未执行，SwiftUI 怎么为 `DynamicProperty.update()` 找到当前 View 的状态？答案是区分“当前 View 的身份”和“它的 body 将产生的子 View 身份”。

递归展开顺序

父节点身份已知
↓ 父 body 产生 ChildView value
运行时把 ChildView 放到父节点的某个结构位置
↓ ChildView 当前节点身份已知
先准备 ChildView 的动态属性，再计算 ChildView.body
↓ body 产生孙节点，并递归重复

根 View 的身份来自 Scene/Window 的托管入口；每个子 View 的当前身份来自父节点的结构或显式 ID。执行某个 View 自己的 body 之前，不需要提前知道它所有后代的身份。

教学伪代码 | 从已知父节点递归展开 View 层级 推导依据：父 body 必须先产生 ChildView value；Child 的动态属性又必须在 Child.body 前准备。由这个公开时序可以推导出逐层递归模型。目的：消除“必须先执行整棵树的 body 才能知道 identity”的循环疑问。边界：只解释公开行为，不代表 Apple 的类型布局、函数名、调度顺序或真实源码。

教学伪代码（非 Apple 源码）

```

func expand(viewValue, currentNode) {
    // currentNode 的 identity 已由父节点确定
    prepareDynamicProperties(
        in: &viewValue,
        context: currentNode.context
    )

    let bodyValue = viewValue.body
    let childDescriptions = enumerateChildren(bodyValue)

    for (slot, childValue) in childDescriptions {
        let childID = identity(
            parent: currentNode.identity,
            structuralSlot: slot,
            viewType: type(of: childValue),
            explicitID: childValue.explicitID
        )
        let childNode = reconcileOrCreate(childID)
        expand(childValue, childNode)
    }
}

```

第三部分 DynamicProperty : body 前的动态输入准备

13. 普通 Property Wrapper 与 DynamicProperty 的区别

普通 Property Wrapper 只是 Swift 语言特性。Swift 不知道它与界面生命周期、Environment 或外部对象有什么关系。`DynamicProperty` 则是 SwiftUI 提供的协议，用于让存储属性在 View 更新周期中获得准备机会。

```

public protocol DynamicProperty {
    mutating func update()
}

```

Apple 的公开文档表述是：SwiftUI 在渲染 View 的 body 前调用 `update()`，以确保该动态属性拥有最新的底层值。`State`、`Binding`、`Environment`、`ObservedObject`、`StateObject`、`FocusState` 等都符合这一体系。

14. `update()` 具体做什么：公开事实与概念模型

公开保证 调用时机在 body 之前；目的是更新动态属性的底层值。具体到每一种包装器，私有实现细节并未完整公开。

为了理解行为，可以把 `update()` 看作“当前 View 节点的动态输入准备阶段”。它不只是为读取刚刚写入的新数值；更重要的是，新的 View value 中的动态属性需要在当前运行时上下文里获得正确的状态位置、环境值或观察关系。

动态属性	概念上的准备工作	数据归属
State	使当前包装器访问到该身份下的状态位置	SwiftUI 管理
Environment	从当前环境链读取键对应的最新值	祖先环境
ObservedObject	使 View 接收对象变化并参与失效	外部对象
Binding	维持指向源状态的 get/set 通道	源状态拥有者
FocusState	连接当前节点与焦点系统	SwiftUI 焦点系统

表中的“使……连接”是行为模型，不表示所有类型都在每轮解除并重新订阅，也不表示 SwiftUI 必定用某张字典按 `(Identity, PropertyIndex)` 查找。

14.1 `mutating update()` 会不会把整个 View struct 改掉

会。从 Swift 值语义看，如果 `\_colorScheme.update()` 修改了 Environment wrapper 的字段，包含它的当前 View value 也发生了变化。但这只是 SwiftUI 已经开始的一轮求值中的准备动作，不是新的状态 mutation。

教学伪代码 | 当前 View 的准备与求值 推导依据：DynamicProperty.update() 的公开调用时机在 body 之前；View 是值类型，mutating update 必然作用于本轮可变工作值。目的：解释 update 修改 wrapper 后为什么不会形成新的用户状态写入循环。边界：真实 SwiftUI 通过动态属性 buffer、字段偏移和图输入工作，不保证使用反射或这个循环。

教学伪代码（非 Apple 源码）

```
func evaluate<V: View>(_ input: V, node: ViewNode) {
    var prepared = input

    for field in dynamicFields(of: prepared) {
        field.installRuntimeContext(from: node)
        field.update()
    }

    let output = prepared.body
    reconcileChildren(of: node, with: output)
}
```

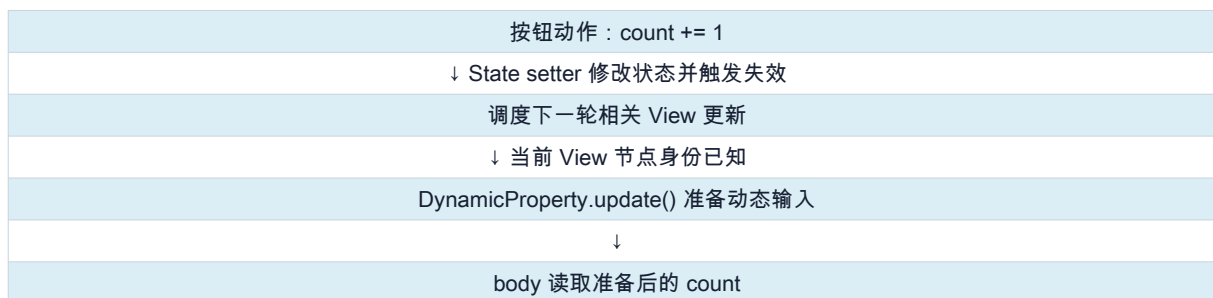
变化	是否发生	是否会自行触发下一轮更新
DynamicProperty 内部字段改变	可能	不会；它是当前求值的准备步骤
View 工作副本的值语义改变	会随嵌套字段改变	不会；View value 不是被观察的数据源
View Identity 改变	不一定	由结构、类型、显式 ID 决定
State/Observation 数据源改变	取决于用户 mutation	会使相应依赖失效

不要形成死循环模型 `update()` 修改的是本轮 body 使用的临时工作值；真正触发 invalidation 的是 State storage、objectWillChange、Observation 属性或 Environment Graph 等动态数据源。

15. 为什么 setter 已经改了 storage，body 前仍要 update

状态写入与下一轮 View value 的准备，是两个不同问题。setter 负责修改状态并触发失效；`update()` 负责在下一轮求值之前，让本轮参与计算的动态属性处于正确运行时上下文。

写入与下一轮读取不是同一职责



16. 自定义 DynamicProperty 实验

```
@propertyWrapper
struct DebugState<Value>: DynamicProperty {
    @State private var value: Value

    init(wrappedValue: Value) {
        _value = State(initialValue: wrappedValue)
    }

    var wrappedValue: Value {
        get { value }
        nonmutating set { value = newValue }
    }

    mutating func update() {
        print("DebugState.update")
    }
}

struct Demo: View {
    @DebugState var count = 0

    var body: some View {
        let _ = print("Demo.body")
        return Button("\(count)") { count += 1 }
    }
}
```

这个实验能观察 `update()` 相对 body 的顺序，但不能通过日志证明 SwiftUI 私有 `State.update()` 内部到底执行了哪些语句。教学代码观察的是协议生命周期，而不是反编译内部实现。

第四部分 状态所有权：Binding、ObservedObject 与 StateObject

17. `@Binding`：把读写能力传给子 View

父子 View 最常见的错误，是双方各保存一份内容相同但彼此独立的 State。真正需要的是单一事实来源：父 View 拥有状态，子 View 只获得读写这份状态的能力。

```
struct Parent: View {
    @State private var name = "Tom"

    var body: some View {
        Child(name: $name)
    }
}

struct Child: View {
    @Binding var name: String

    var body: some View {
        TextField("Name", text: $name)
    }
}
```

写法	类型	角色
name	String	Binding 背后的当前值
_name	Binding<String>	编译器生成的包装器存储
\$name	Binding<String>	继续把同一读写通道传给控件或后代

如果手写初始化器，参数是 `Binding<String>`，因此赋给包装器本身，而不是赋给 wrapped value：

```
struct Child: View {
    @Binding var name: String

    init(name: Binding<String>) {
        self._name = name
    }
}
```

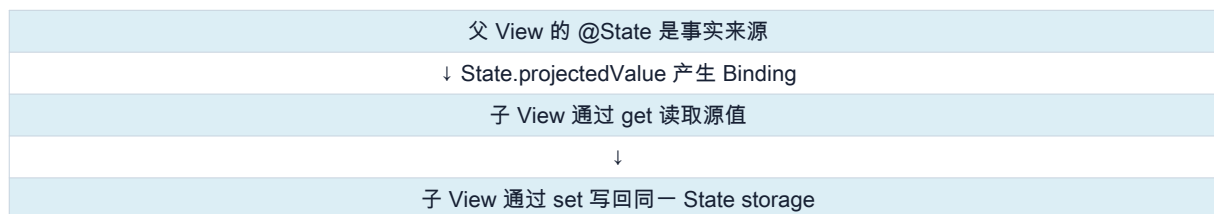
17.1 Binding 没有独立 Storage

用于入门的模型是 getter 与 setter。真实 Binding 还集成了 transaction、location 与动态成员查找，但“不拥有值”这个语义不会改变。

```
struct SimpleBinding<Value> {
    let getValue: () -> Value
    let setValue: (Value) -> Void

    var wrappedValue: Value {
        get { getValue() }
        nonmutating set { setValue(newValue) }
    }
}
```

一份状态，两处访问

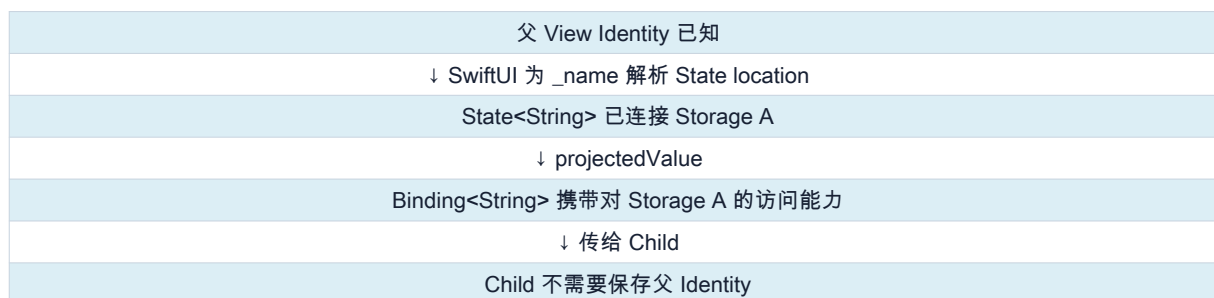


因此 `Child(name: name)` 传的是当次 View value 中的普通值；`Child(name: \$name)` 传的是双向读写通道。只读子 View 应优先接收普通值，确实需要修改来源时才接收 Binding。

17.2 Binding 如何连接到正确的 State storage

Binding 通常不保存父 View Identity，也不会每次读写都沿父树重新查找。顺序是：SwiftUI 先利用父节点 Identity 和动态属性位置，让 `@State` 连接正确的状态位置；随后 `\$state` 从这个已经解析的 location 产生 Binding。

Identity 用于定位；Binding 使用定位结果



自定义 Binding 可以完全脱离 View Identity，这也证明 Identity 不是 Binding 的通用组成部分：

```
let binding = Binding<Int>{
    get: { box.value },
    set: { box.value = $0 }
}
```

教学伪代码 | 从 State location 投影 Binding 推导依据：State.projectedValue 的公开类型是 Binding；Binding 自身不拥有源值，自定义 Binding 也只要求 get/set。目的：解释 Binding 为什么无需保存父 View Identity，却能持续访问父 State 的正确存储位置。边界：真实 Binding 可能封装 Location、Transaction 和动态成员信息；不保证以两个 Swift 闭包存储。

教学伪代码（非 Apple 源码）

```
func projectBinding<Value>(
    from location: StateLocation<Value>
) -> Binding<Value> {
    Binding(
        get: {
            location.read()
        },
        set: { newValue, transaction in
            location.write(
                newValue,
                transaction: transaction
            )
        }
    )
}
```

生命周期规则 Binding 不会延长源状态的语义生命周期。不要把局部 View 产生的 Binding 长期保存到全局对象；需要长期存在的数据，应由更稳定的 Model 或 Store 拥有。

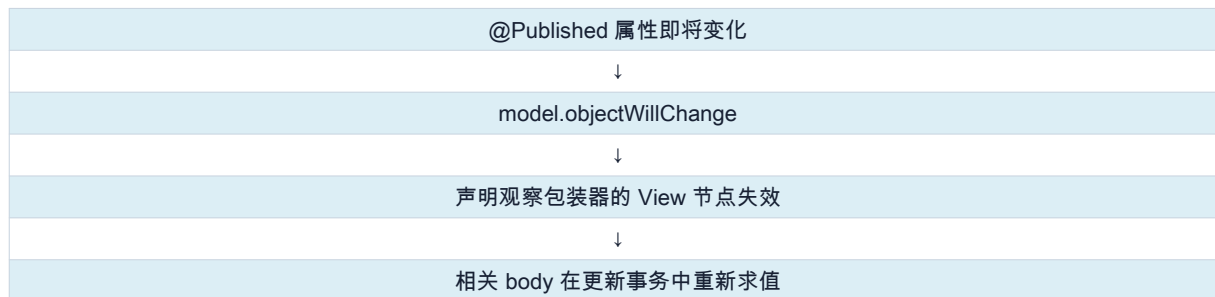
18. 经典对象系统：ObservableObject、Published 与观察包装器

普通 class 的内部属性变化不会自动使 SwiftUI 更新。经典系统把职责拆为：`ObservableObject` 提供 `objectWillChange`，`@Published` 在属性变化前发送对象级通知，View 中的观察包装器接收通知并使当前 View 失效。

```
final class ProfileModel: ObservableObject {
    @Published var name = "Tom"
    @Published var age = 20

    init(name: String = "Tom", age: Int = 20) {
        self.name = name
        self.age = age
    }
}
```

经典对象级通知



粒度边界 `objectWillChange` 表达“这个对象将变化”，而不是“只有某个字段的读者变化”。因此只显示 name 的 View，也可能因为同一对象的 age 变化而失效。

18.1 `@Published` 的语言展开与通知链

`@Published` 本身也是 Property Wrapper。下面的展开只表达语言层结构；Combine 如何把多个 Published 属性关联到默认 `objectWillChange` publisher，属于框架实现。

```
final class ProfileModel: ObservableObject {
    @Published var name = "Tom"
}

// 语言层可以近似阅读为：
private var _name = Published<String>(initialValue: "Tom")

// `name` 的访问由 Published 的 enclosing-instance
// subscript 路由到拥有者和 _name 存储。
// `$name` 访问 _name.projectedValue，
// 类型是 Published<String>.Publisher
```

更具体的 SDK 细节 Published 的 wrappedValue 在当前 Combine interface 中标为 unavailable，并提供 `\_enclosingInstance` static subscript。也就是说，编译器访问 class 的 Published 属性时，可以把“拥有该属性的对象”“包装值 KeyPath”“包装器存储 KeyPath”一起交给 wrapper，而不是只调用一个孤立的 `\_name.wrappedValue`。上面的简单展开只是第一层语言模型。

```
// 从当前 Combine interface 简化
static subscript<Owner>(
    _enclosingInstance object: Owner,
    wrapped wrappedKeyPath:
    ReferenceWritableKeyPath<Owner, Value>,
    storage storageKeyPath:
    ReferenceWritableKeyPath<Owner, Published<Value>>
) -> Value { get set }
```

这个 enclosing-instance 入口解释了为什么 Published 只能用于 class 属性，也为 wrapper 访问拥有者的 objectWillChange 提供了机制基础。Combine 如何发现/合成 ObservableObjectPublisher 仍是框架细节，不应把上面的 subscript 直接等同于完整发送源码。

教学伪代码 | Published setter 如何形成对象级通知 推导依据：当前 Combine interface 暴露 enclosing-instance subscript；ObservableObject 暴露 objectWillChange；公开行为是在属性改变前发送对象变化。目的：解释为什么改变任一 Published 字段，ObservableObject 观察的是“对象将变化”，而非自动获得字段级依赖。边界：发送与赋值的精确内部顺序、publisher 存放位置及去重策略未公开；这里仅表达对象通知和值流两种职责。

教学伪代码（非 Apple 源码）

```
static subscript(
    _enclosingInstance owner,
    wrapped valueKeyPath,
    storage wrapperKeyPath
) -> Value {
    get {
        owner[wrapperKeyPath].currentValue
    }
    set {
        owner.objectWillChange.send()
        owner[wrapperKeyPath].currentValue = newValue
        owner[wrapperKeyPath].valuePublisher.send(newValue)
    }
}
```

入口	发布什么	典型用途
model.objectWillChange	对象即将发生变化；默认不携带字段值	SwiftUI 观察整个对象
model.\$name	name 属性的值流	Combine 管道、debounce、测试
model.name	当前 String 值	普通读取与写入

默认 ObservableObject publisher 会在属性改变前发送。因此订阅 `objectWillChange` 的闭包里立即读取属性，可能仍看到旧值；SwiftUI 使用这个事件的目的不是读取新值，而是把相关 View 标记为需要在之后的更新周期重新求值。

18.2 不使用 Published 时如何手动通知

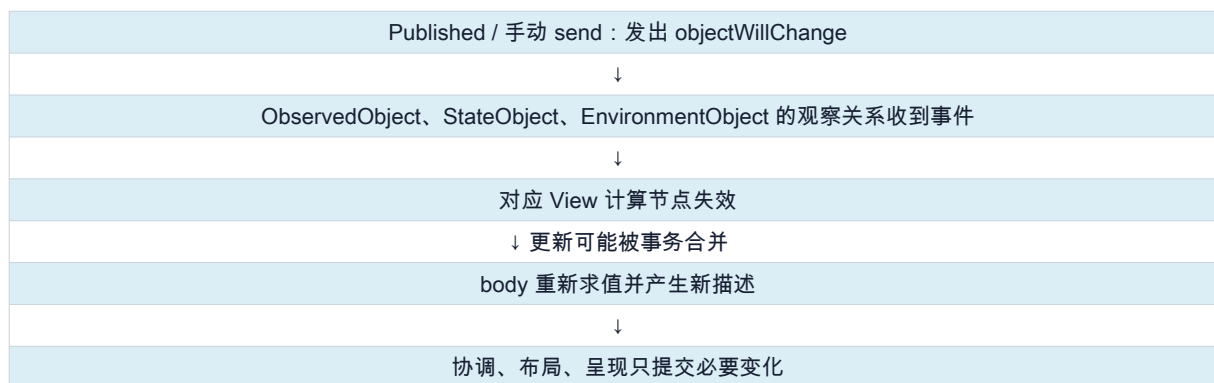
```
final class ManualModel: ObservableObject {
    let objectWillChange = ObservableObjectPublisher()

    var name = "Tom" {
        willSet {
            objectWillChange.send()
        }
    }
}
```

这能说明经典系统真正要求的是 `objectWillChange` publisher，而不是必须使用 Published。手动发送时要确保 mutation 与 UI 隔离符合并发规则；面向界面的模型通常声明为 `@MainActor`，避免从后台线程直接改变 UI 观察的数据。

18.3 通知、失效与重绘是三个阶段

对象通知不会直接调用某个 UILabel.setText



19. `@ObservedObject` : 观察外部拥有的对象

```
struct ProfileContent: View {
    @ObservedObject var model: ProfileModel

    var body: some View {
        Text(model.name)
    }
}
```

`@ObservedObject` 的所有权语义是：对象由父级、协调器或依赖容器创建；当前 View 只接收引用并观察它。它当然会在包装器中持有正在使用的引用，但不会像 StateObject 那样为对象建立与当前 View Identity 绑定的稳定所有权。

19.1 先看公开类型表面

按公开接口简化，省略 actor 与可用性标注

```
@propertyWrapper
struct ObservedObject<ObjectType>: DynamicProperty
where ObjectType: ObservableObject {
    init(wrappedValue: ObjectType)

    var wrappedValue: ObjectType { get set }
    var projectedValue: Wrapper { get }

    @dynamicMemberLookup
    struct Wrapper {
        subscript<Subject>(
            dynamicMember keyPath:
                ReferenceWritableKeyPath<ObjectType, Subject>
        ) -> Binding<Subject> { get }
    }
}
```


这里能确认三件事：ObservedObject 是 DynamicProperty；wrappedValue 返回对象引用；projectedValue 不是对象本身，而是能通过引用可写 KeyPath 产生 Binding 的 Wrapper。订阅 token、失效回调等并没有作为公开字段暴露。

当前 SDK 能看到什么 在本讲义校验所用的 Xcode 26.5 SDK Swift interface 中，ObservedObject 还暴露了内部 `\_seed: Int` 和 `\_makeProperty(...)` 声明。但仅凭这些符号无法推出订阅对象、图节点和失效回调的真实布局，因此正文只把订阅过程画成行为模型。

19.2 编译器如何展开 View 中的声明

```
struct ProfileContent: View {
    @ObservedObject var model: ProfileModel
}

// 近似语言展开：
struct ProfileContent: View {
    private var _model: ObservedObject<ProfileModel>

    var model: ProfileModel {
        _model.wrappedValue
    }

    // 投影访问 `$model`
    // 等价于使用 _model.projectedValue

    init(model: ProfileModel) {
        self._model = ObservedObject(wrappedValue: model)
    }
}
```

不要把展开写成存储三个属性 `model` 与 `\$model` 是访问语法，不是另外复制了两份对象。真正的包装器存储是 `\_model`；wrappedValue、projectedValue 都从它取得。

19.3 一个用于理解的订阅模型

SwiftUI 私有实现并未公开，下面只解释可观察行为，不能当作源码。关键是订阅必须属于运行时 View 节点，而不是临时 View value 自己长期保存一个 AnyCancellable。

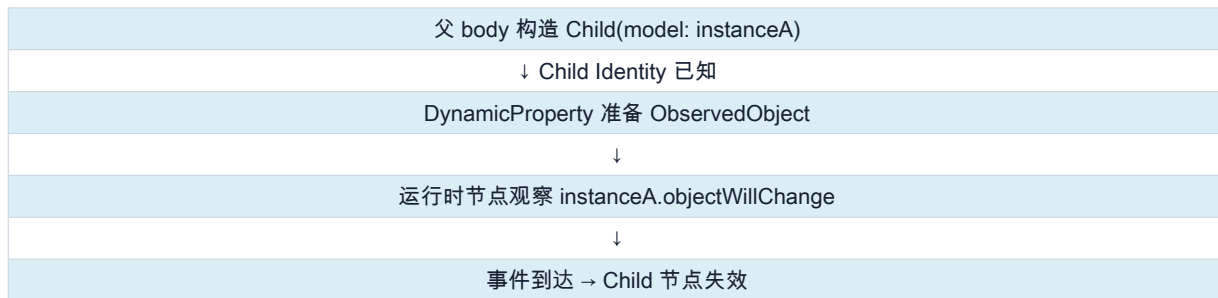
教学伪代码 | ObservedObject 安装和切换订阅 推导依据：ObservedObject 遵循 DynamicProperty；官方定义其订阅 ObservableObject 并在变化时使 View 失效；输入对象引用允许随新 View value 改变。目的：解释订阅为什么跟随逻辑 View 节点，以及对象 A 切换为 B 时为何要替换观察关系。边界：订阅 token 的类型、保存位置、去重和事务合并均未公开；函数名和闭包结构仅为教学表达。

教学伪代码（非 Apple 源码）

```
func prepareObservedObject<Object: ObservableObject>(
    wrapper: inout ObservedObject<Object>,
    node: ViewNode
){
    let object = wrapper.wrappedValue

    node.replaceSubscriptionIfNeeded(for: object) {
        object.objectWillChange.sink {
            node.invalidate()
        }
    }
}
```

订阅跟随逻辑节点和当前输入对象



问题	@ObservedObject 的答案
谁创建对象	外部所有者
谁决定对象何时释放	外部强引用关系
当前 View 做什么	订阅 objectWillChange 并使用对象
对象改变时	当前观察者 View 成为失效入口

不应在需要自有生命周期的 View 中写 `@ObservedObject private var model = Model()`；View value 重新初始化时可能创建新对象，导致状态、任务和订阅重置。

19.4 为什么在 ObservedObject 上可以写 `\$model.name`

```
struct ProfileEditor: View {
    @ObservedObject var model: ProfileModel

    var body: some View {
        TextField("Name", text: $model.name)
    }
}

// 概念上：
let nameBinding: Binding<String> =
    _model.projectedValue[dynamicMember: \.name]
```

Binding 的 getter/setter 直接读写对象属性。setter 修改 `model.name` 后，Published 再发出 objectWillChange；Binding 负责写值，ObservedObject 负责使 View 响应对象通知，两者职责不能混为一谈。

19.5 同一 View Identity 下更换输入对象

```
struct Host: View {
    @State private var useA = true
    @StateObject private var a = ProfileModel(name: "A")
    @StateObject private var b = ProfileModel(name: "B")

    var body: some View {
        ProfileContent(model: useA ? a : b)
    }
}
```

ProfileContent 的结构位置和类型没有改变，因此它自己的局部 State 可以继续存在；但 `@ObservedObject` 的输入引用从 a 变为 b，运行时必须停止把 a 的通知当作当前依赖，并开始观察 b。ObservedObject 的“可替换输入”正是它与 StateObject 所有权语义不同的地方。

19.6 两个常见反例

```
// 反例一：每次父 body 求值都可能创建新对象
struct WrongChild: View {
    @ObservedObject private var model = ProfileModel()

    var body: some View {
        Text(model.name)
    }
}

// 反例二：普通 let 不观察 objectWillChange
struct ReadOnlyReference: View {
    let model: ProfileModel
    var body: some View { Text(model.name) }
}
```

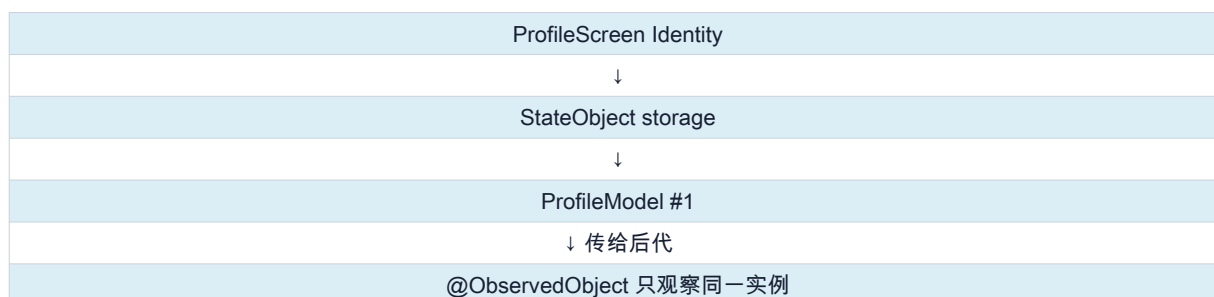
第一个写法把“创建”和“观察”混在一起；第二个写法可以显示传入对象，但在经典 ObservableObject 系统里，model 内部 Published 变化不会直接使这个 View 失效。若父级因其他原因重算并传入新 View value，屏幕可能碰巧更新，这不能替代正确观察。

20. `@StateObject`：在 View Identity 下拥有对象

```
struct ProfileScreen: View {
    @StateObject private var model = ProfileModel()

    var body: some View {
        ProfileContent(model: model)
    }
}
```

一个所有者，可以有多个观察者



Identity 不变时，新 ProfileScreen value 继续取得 `ProfileModel #1`；Identity 消失时，SwiftUI 结束这份 StateObject storage。若没有其他强引用，对象才会随后 deinit。

包装器	所有权	观察机制	典型位置
@StateObject	当前 View Identity 拥有	objectWillChange	页面或 Feature 根
@ObservedObject	外部拥有	objectWillChange	显式接收对象的子 View
@EnvironmentObject	祖先注入方拥有	objectWillChange	跨多层的后代消费者

20.1 完整父子协作示例

```
@MainActor
final class SettingsModel: ObservableObject {
    @Published var userName = ""
    @Published var notificationsEnabled = true
}

struct SettingsScreen: View {
    @StateObject private var model = SettingsModel()

    var body: some View {
        SettingsForm(model: model)
    }
}

struct SettingsForm: View {
    @ObservedObject var model: SettingsModel

    var body: some View {
        TextField("Name", text: $model.userName)
        Toggle(
            "Notifications",
            isOn: $model.notificationsEnabled
        )
    }
}
```

动作	StateObject 所有者	ObservedObject 消费者
首次进入	创建 SettingsModel 并拥有	接收同一引用并观察
父 body 重算	Identity 不变则复用对象	获得同一引用的新 wrapper value
表单写属性	对象内容改变	Binding 写值并收到 objectWillChange
页面移除	结束 identity storage	消费者节点及其观察关系结束

这是一种常见但不是强制的分层：Feature 根负责拥有生命周期，叶子 View 负责展示和编辑。若中间 View 只负责透传 model，可以使用普通属性传递引用；真正读取 Published 数据的经典 View 再声明 ObservedObject。

20.2 公开类型表面与编译器展开

按公开接口简化

```
@propertyWrapper
struct StateObject<ObjectType>: DynamicProperty
where ObjectType: ObservableObject {
    init(
        wrappedValue thunk:
        @autoclosure @escaping () -> ObjectType
    )

    var wrappedValue: ObjectType { get }
    var projectedValue:
        ObservedObject<ObjectType>.Wrapper { get }
}
```

```

struct ProfileScreen: View {
    @StateObject private var model = ProfileModel()
}

// 近似语言展开：
struct ProfileScreen: View {
    private var _model: StateObject<ProfileModel> =
        StateObject(wrappedValue: ProfileModel())

    var model: ProfileModel {
        _model.wrappedValue
    }

    // 投影访问 `$model`
    // 等价于使用 _model.projectedValue
}

```

表面上仍写 `ProfileModel()`，但由于初始化器参数是 autoclosure，表达式被编译器包装为 `() -> ProfileModel`，而不是在每次构造 StateObject wrapper 时先无条件执行。projectedValue 复用 ObservedObject.Wrapper，所以 `\$model.name` 的动态成员 Binding 机制与 ObservedObject 一致。

20.3 SDK interface 直接显示的两阶段 Storage

这一点比纯推测更具体：本讲义使用的 Xcode 26.5 SDK Swift interface 中，可以看到 StateObject 的 internal Storage 枚举。它不是稳定的公开 API，但能验证“先保存闭包，安装后转为 ObservedObject”这条解释路线。

```

// 摘取并简化自 Xcode 26.5 SDK swiftinterface
internal enum Storage {
    case initially(() -> ObjectType)
    case object(ObservedObject<ObjectType>)
}

internal var storage: Storage

init(
    wrappedValue thunk:
    @autoclosure @escaping () -> ObjectType
){
    storage = .initially(thunk)
}

```

`initially` 保存创建闭包；`object` 保存 ObservedObject wrapper。公开的 wrappedValue/projectedValue 会根据当前 storage 提供对象和属性 Binding。SwiftUI 何时、通过哪个图节点把 initially 安装为 object，仍由 `\_makeProperty` 等私有运行时逻辑完成。

创建闭包是候选输入，持久对象位于身份存储

新的 View value
↓ StateObject.storage = .initially(thunk)
SwiftUI 根据 View Identity + 属性位置安装动态属性
↓ 已存在 ↓ 不存在
复用已安装对象 执行 thunk()
↓ 转为 .object(ObservedObject)
wrappedValue 返回该身份下的稳定对象

Storage 枚举能解释 wrapper 自身的两个阶段，但不能单独解释跨 View value 的持久性。真正让对象跟随 Identity 延续的仍是 SwiftUI 动态属性 buffer / 图节点所管理的位置；不要误读为“每次新建 StateObject struct 自己就能记住旧对象”。

教学伪代码 | StateObject 首次安装与后续复用 推导依据：公开初始化器保存 autoclosure；当前 SDK interface 显示 `.initially(thunk)` 与 `.object(ObservedObject)`；官方文档保证每个容器实例只创建一次。目的：把 wrapper 的两阶段 storage 与 View Identity 下的持久 location 串起来。边界：真实代码不会公开 `initialThunk()` 或这些 location 类型；安装还涉及 actor、图输入、字段偏移和框架优化。

教学伪代码（非 Apple 源码）

```

func installStateObject<Object: ObservableObject>(
    wrapper: inout StateObject<Object>,
    node: ViewNode,
    field: FieldID
) {
    if let location = node.objectLocations[field] {
        wrapper.storage = .object(location.observed)
        return
    }

    let object = wrapper.initialThunk()
    let observed = ObservableObject(wrappedValue: object)
    let location = ObjectLocation(observed: observed)

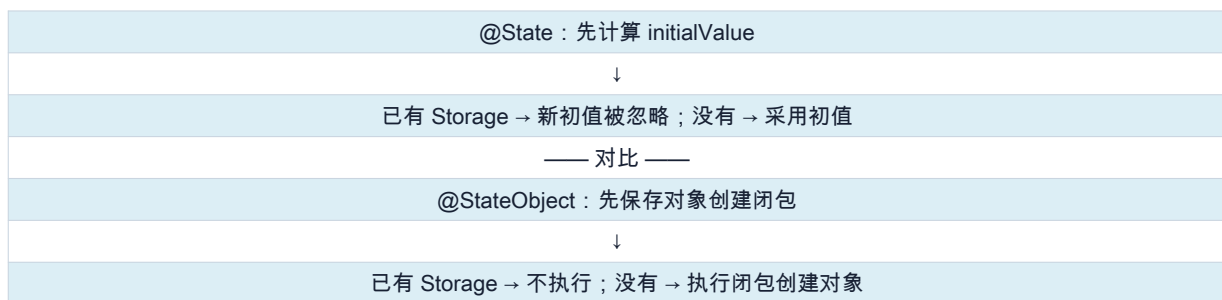
    node.objectLocations[field] = location
    wrapper.storage = .object(observed)
    installObjectChangeInvalidation(location, on: node)
}

```

20.4 State 与 StateObject 的初始化为什么不同

‘State’ 直接接收已经计算出的 ‘Value’；‘StateObject’ 的初始化参数概念上是 ‘@autoclosure @escaping () -> ObjectType’。后者让 SwiftUI 能先查找当前 Identity 是否已有对象，再决定是否执行创建表达式。

先计算值，还是先判断需不需要对象



对象具有引用 Identity，初始化还可能建立订阅、计时器或昂贵资源。如果 StateObject 先无条件创建对象再丢弃，就会产生严重副作用。‘@autoclosure’ 让调用处仍能自然地写 ‘Model()’；‘@escaping’ 则允许把创建表达式保存到初始化器返回之后。

20.5 外部初始化参数只在第一次采用

```

struct UserScreen: View {
    @StateObject private var model: UserModel

    init(userID: String) {
        _model = StateObject(
            wrappedValue: UserModel(userID: userID)
        )
    }
}

```

当 View Identity 不变时，后来传入的新 ‘userID’ 不会自动替换已经存在的 model。若它表示同一对象的新输入，应显式调用加载方法；若它表示全新业务实体，应重新设计所有权，必要时才用稳定业务 ID 改变 Identity。

20.6 四组 StateObject 生命周期实验

20.6.1 父 State 变化不会重建对象

```
@MainActor
final class LoggedModel: ObservableObject {
    let id = UUID()

    init() { print("MODEL INIT", id) }
    deinit { print("MODEL DEINIT", id) }
}

struct Host: View {
    @State private var refreshCount = 0

    var body: some View {
        VStack {
            Button("Parent update") { refreshCount += 1 }
            OwnedChild()
        }
    }
}

struct OwnedChild: View {
    @StateObject private var model = LoggedModel()

    var body: some View {
        Text(model.id.uuidString)
    }
}
```

让 LoggedModel.init/deinit 打印实例 ID。点击按钮会重算 Host.body 并再次构造 OwnedChild value，但 OwnedChild 的结构身份没变，StateObject 应继续返回同一对象。

20.6.2 同一位置写两个 Child，得到两个对象

```
VStack {
    OwnedChild() // 结构位置 0
    OwnedChild() // 结构位置 1
}
```

类型相同不代表 Identity 相同。两个结构位置各有自己的 StateObject location，因此各创建一个 LoggedModel。StateObject 不是按类型缓存，更不是单例。

20.6.3 条件移除会结束所有权

```
if showDetails {
    OwnedChild()
}
```

showDetails 变为 false 后，该分支离开树，StateObject storage 生命周期结束。若没有外部强引用，model 随后 deinit；再次显示时是新身份生命周期和新对象。释放的精确时刻不应作为同步业务契约。

20.6.4 `.id(userID)` 表达业务实体切换

```
UserScreen(userID: selectedID)
    .id(selectedID)
```

selectedID 改变会让 SwiftUI 把整棵 UserScreen 子树视为新实体，于是 StateObject 使用新创建闭包建立模型。它同时也会重置该子树的 State、任务、焦点等所有身份资源，因此只能在“逻辑实体确实变化”时使用。

20.7 iOS 17 Observation 为什么改用 State

Observation 把“引用所有权”和“内部属性观察”拆开：`@State` 在 Identity 下保存 `@Observable` 引用；`@Observable` 生成属性访问追踪。因此不再需要 StateObject 对 `objectWillChange` 的 Combine 订阅。

```
@Observable
final class Library {
    var books: [Book] = []
}

struct LibraryView: View {
    @State private var library = Library()
}
```

真实代价 `@State private var library = Library()` 在每次初始化该 View value 时都会计算默认表达式，可能产生随后被忽略的临时对象。让 Model.init 保持便宜且无副作用；昂贵创建可用可选 State 加 `.task` 延迟，或交给更稳定的外部所有者。

昂贵 Model 的延迟创建

```
struct LibraryView: View {
    @State private var library: Library?

    var body: some View {
        Group {
            if let library { LibraryContent(model: library) }
            else { ProgressView() }
        }
        .task {
            guard library == nil else { return }
            library = Library()
        }
    }
}
```

第五部分 树状上下文：Environment 与 EnvironmentObject

21. `@Environment`：读取当前 View 所处的上下文

Environment 用于主题、Locale、字体、Scene phase、dismiss action 或跨多层共享的依赖。它不是全局单例：不同子树可以同时拥有不同值，离当前 View 最近的覆盖生效。

```
struct Child: View {
    @Environment(\.colorScheme) private var colorScheme

    var body: some View {
        Text(colorScheme == .dark ? "Dark" : "Light")
    }
}
```

21.1 EnvironmentValues 是类型安全的异构存储

教学上可以把 `EnvironmentValues` 想成按 Key 类型索引的字典。自定义 Key 提供默认值，`EnvironmentValues` 的计算属性再提供 KeyPath 入口。真实容器、共享和缓存策略属于 SwiftUI 私有实现。

```
private struct AppThemeKey: EnvironmentKey {
    static let defaultValue: AppTheme = .light
}

extension EnvironmentValues {
    var appTheme: AppTheme {
        get { self[AppThemeKey.self] }
        set { self[AppThemeKey.self] = newValue }
    }
}

// 读取
@Environment(\.appTheme) private var theme
```


层次	作用
AppThemeKey.self	概念上的类型 Key，并提供 defaultValue
EnvironmentValues.appTheme	通过 KeyPath 暴露类型安全访问
@Environment(\.appTheme)	保存读取路径，body 前解析当前值
.environment(\.appTheme, value)	为该 View 子树建立局部覆盖

21.2 保存与查找：更像向下传播，而不是每次向上遍历

公开语义可以说“从当前 View 向上找到最近祖先覆盖”；更接近高效实现的模型是：父节点环境已经解析，Environment modifier 从它派生一个局部版本，再向后代传播。于是当前节点直接读取已经覆盖完成的环境。

最近覆盖是环境派生顺序的结果



逻辑上每个节点拥有环境快照；物理上 SwiftUI 可以使用 copy-on-write、结构共享或 AttributeGraph 属性避免复制整个大型字典。兄弟分支获得不同的逻辑环境版本，因此互不污染。

教学伪代码 | 环境派生与最近覆盖查找 推导依据：Environment modifier 只影响后代；同一 key 的最近祖先覆盖生效；没有覆盖时使用 EnvironmentKey.defaultValue。目的：解释逻辑上的树状作用域，同时避免把 Environment 误解为全局可变字典。边界：真实实现可以用持久数据结构、图属性或缓存直接传播解析后的快照，不一定逐级向上循环。

教学伪代码（非 Apple 源码）

```
struct EnvironmentFrame {
    let parent: EnvironmentFrame?
    var overrides: [KeyID: Any]
}

func deriveEnvironment<Value>(
    parent: EnvironmentFrame,
    key: EnvironmentKey<Value>,
    value: Value
) -> EnvironmentFrame {
    EnvironmentFrame(
        parent: parent,
        overrides: [key.id: value]
    )
}

func resolve<Value>(
    _ key: EnvironmentKey<Value>,
    from frame: EnvironmentFrame
) -> Value {
    frame.nearestOverride(for: key.id)
    ?? key.defaultValue
}
```

21.3 `@Environment.update()` 与 View 工作副本

View.init 时包装器只知道 KeyPath，还不知道 View 最终位于哪条树路径。SwiftUI 在 body 前准备当前节点环境，再以 mutating update 把本轮正确输入交给 View 工作副本。这个内部字段变化不会制造新的 Identity，也不会再次触发 invalidation。

Environment 是 body 的动态输入

父节点产生新的 Child value
↓ 当前 Child 节点与环境已知
@Environment.update() 解析 KeyPath
↓ 修改本轮 View 工作副本
body 读取准备后的环境值

教学伪代码 | Environment wrapper 在 body 前取得当前值 推导依据：Environment 是 DynamicProperty；初始化时只持有读取描述，当前 View 所在的环境要到运行时安装后才确定。目的：解释 mutating update 改变的是本轮工作副本，而不是写入一个新的业务状态。边界：公开 Environment 的字段布局和缺省状态并非如此；cachedValue/context 都是为说明调用时机而虚构。

教学伪代码（非 Apple 源码）

```
struct TeachingEnvironment<Value>: DynamicProperty {
    let keyPath: KeyPath<EnvironmentValues, Value>
    var cachedValue: Value
    var context: DynamicPropertyContext?

    mutating func update() {
        guard let context else { return }
        cachedValue =
            context.environment[keyPath: keyPath]
    }

    var wrappedValue: Value {
        cachedValue
    }
}
```

init 中不要读 Environment View.init 尚未进入 SwiftUI Runtime，环境还没有从父节点传播。此时只能得到默认或未安装状态的值；应在 body、task 等已安装上下文中使用。

21.4 iOS 17：把 Observable 对象放入 Environment

```
@Observable final class Session {
    var userName = ""
}

@State private var session = Session()

RootView()
    .environment(session)

struct ProfileView: View {
    @Environment(Session.self) private var session
}
```

这种形式按 `Session.self` 查找对象。非可选读取在环境缺失时会报告运行时错误；可选声明 `Session?` 可在缺失时得到 nil。需要属性 Binding 时，在 body 中用 `@Bindable` 包装取得的对象。

22. `@EnvironmentObject`：环境查找加对象级观察

经典系统中，祖先用 `.environmentObject(model)` 把 `ObservableObject` 引用按类型写入子树环境；后代的 `@EnvironmentObject` 按类型取得最近实例，并订阅 `objectWillChange`。

```

@main
struct MyApp: App {
    @StateObject private var session = SessionModel()

    var body: some Scene {
        WindowGroup {
            RootView()
                .environmentObject(session)
        }
    }
}

struct ProfileView: View {
    @EnvironmentObject private var session: SessionModel
}

```

Environment 查找 + ObservedObject 式监听

@StateObject 创建并拥有 SessionModel
↓ .environmentObject(session)
环境条目：SessionModel.self → 实例 #1
↓ 沿子树传播并允许就近覆盖
@EnvironmentObject 按类型取得 #1
↓ 订阅 objectWillChange
对象变化使当前消费者 View 失效

它不创建对象，也不拥有对象的业务生命周期。环境中找不到非可选的对应类型时会运行时失败；Preview、独立 UIHostingController 和新的 View root 都必须重新注入。

教学伪代码 | EnvironmentObject = 类型查找 + 对象观察 推导依据：environmentObject(·) 把 ObservableObject 提供给后代；EnvironmentObject 按类型取得对象，并在对象变化时使消费者 View 更新。目的：把“Environment 负责找到谁”和“ObservedObject 式监听负责何时更新”拆成两个步骤。边界：真实环境 key、错误形式、订阅复用和 wrapper 字段均未公开；按类型查找是公开使用语义。

教学伪代码（非 Apple 源码）

```

func prepareEnvironmentObject<Object: ObservableObject>(
    wrapper: inout EnvironmentObject<Object>,
    node: ViewNode,
    environment: EnvironmentValues
){
    guard let object = environment[Object.self] else {
        runtimeError(
            "Missing EnvironmentObject<\(Object.self)>"
        )
    }

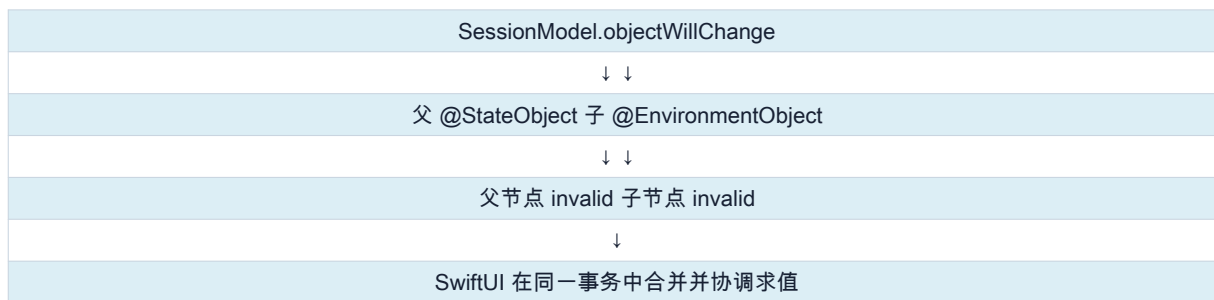
    wrapper.object = object
    prepareObservedObject(
        object: object,
        invalidating: node
    )
}

```

22.1 父子是否都会重算

若父 View 用 `@StateObject` 持有对象，子 View 用 `@EnvironmentObject` 使用同一对象，那么双方都独立订阅 `objectWillChange`，对象变化时双方都可能成为失效入口。不是子 View 向上通知父 View，而是同一个 publisher 同时通知两个观察者。

多个观察者，不是向上传播



如果中间 Provider 只是 `let model: Model`，它没有经典观察包装器，因此对象内部变化不会直接使该 Provider 失效；子 View 的 EnvironmentObject 仍会收到通知。

22.2 同类型冲突与使用边界

EnvironmentObject 以 `ObjectType.self` 作为 Key，不能自然区分 personalSession 与 businessSession 两个同类型角色。需要多个同类型对象时，优先使用显式参数、不同包装类型或重新划分模型边界。

场景	更合适的选择
直接父子传递对象	@ObservedObject，依赖在调用处可见
子 View 只需修改一个值	@Binding，缩小能力和耦合
经典模型跨很多层共享	@EnvironmentObject
iOS 17 @Observable 模型跨层共享	@Environment(Model.self)
同类型两个业务角色	显式参数或不同类型，不依赖类型查找

22.3 两代环境对象体系

职责	经典 ObservableObject	iOS 17 Observation
模型声明	ObservableObject + @Published	@Observable + 普通 var
所有者	@StateObject	@State
注入	.environmentObject(model)	.environment(model)
读取	@EnvironmentObject	@Environment(Model.self)
变化粒度	objectWillChange 对象级	body 实际读取的属性
生成 Binding	\$model.property	@Bindable 后用 \$model.property

第六部分 依赖追踪、AttributeGraph 与 Observation

23. 依赖追踪回答“谁依赖谁”

Identity 回答“这个节点是谁”；依赖追踪回答“这个节点的计算读取了哪些动态输入”。两者一起才能决定：状态是否延续，以及数据变化后哪里需要重新求值。

问题	主要机制
这个 View 还是上次那个节点吗？	Identity
它的 State 是否继续存在？	Identity + lifetime
这个 body 读取了哪些动态值？	Dependency tracking
某个值变化后谁成为更新入口？	依赖图 + invalidation
哪些像素最终改变？	协调、布局与渲染

24. 依赖不是通过静态扫描 `Text("\(count)")` 得到的

一个实用心智模型是：SwiftUI 在运行时求值 body 时处于依赖收集上下文；动态数据的 getter 被访问后，运行时记录“当前计算读取了这个输入”。条件分支让依赖集合可以随执行路径动态变化。

```
var body: some View {
    if model.isEnabled {
        Text(model.name)
    } else {
        Text("Disabled")
    }
}
```

在 `isEnabled == false` 的这一轮，body 会读取 `isEnabled`，但不会读取 `name`。当 `isEnabled` 变为 true 后，body 重新求值并开始读取 `name`，新的依赖关系才建立。

粒度要谨慎表述 对 `@State` 来说，最安全的理解是“读取该 State 的 View 计算节点依赖它”。不要武断地说 SwiftUI 一定直接建立 `count → Text` 的公开边，因为自定义 View 的 body 求值边界与框架原语的内部节点并未完全公开。

教学伪代码 | 运行时读取收集依赖 推导依据：Observation 明确通过属性访问形成依赖；State、Environment 等动态输入的行为也表现为“读取者在输入变化后失效”。目的：解释依赖为何随 if 分支动态变化，而不是编译器静态扫描所有可能出现的属性。边界：真实实现必须处理嵌套求值、线程/actor、事务、边复用和图缓存，不会依赖一个全局可变变量。

教学伪代码（非 Apple 源码）

```
var currentComputation: ComputationNode?

func evaluate(_ node: ComputationNode) {
    let previous = currentComputation
    currentComputation = node
    node.clearPreviousDependencies()

    node.value = node.compute()

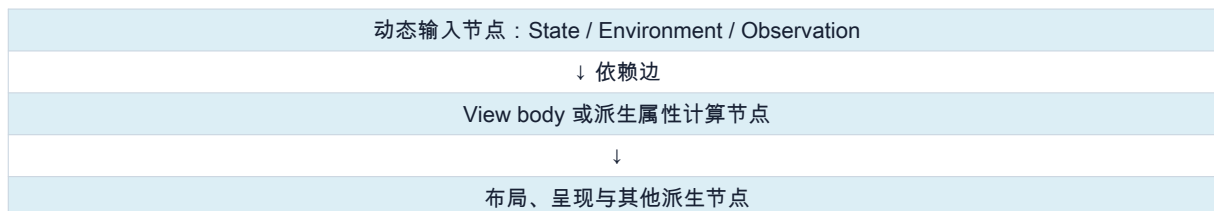
    currentComputation = previous
}

func read<Input>(_ input: InputNode) -> Input.Value {
    if let reader = currentComputation {
        graph.addEdge(from: input, to: reader)
    }
    return input.value
}
```

25. AttributeGraph 是什么，又不是什么

AttributeGraph 是 Apple 系统中的私有依赖计算框架，SwiftUI 使用它管理大量属性之间的关系。公开演讲与可观察行为支持“图节点、依赖边、失效传播、按需重新求值”的总体心智模型，但节点布局、缓存策略和具体算法不是稳定公开 API。

概念上的依赖图



可以合理确认	不应写成确定事实
SwiftUI 根据依赖决定何时更新相关视图。	内部就是一个 `[ViewIdentity: StateLocation]` 字典。
State、Environment、Observation 能驱动失效。	每次 getter 都直接把 Text 节点注册到某个公开图。
依赖图不仅服务于 @State。	所有布局与渲染节点的精确类型、边和调度算法。

可以合理确认	不应写成确定事实
body 可能频繁求值，但最终更新更精细。	只要 body 重算就一定重建所有 UIView。

教学伪代码 | mutation、失效传播与事务合并 推导依据：SwiftUI 会批处理状态变化，依赖图能把输入变化传播到相关计算；body 重算与最终呈现更新是不同阶段。目的：解释连续多次 mutation 为什么不一定产生同样次数的屏幕提交。边界：AttributeGraph 的节点类型、拓扑策略、事务时机和线程模型都是私有实现；这里只表达阶段关系。

教学伪代码（非 Apple 源码）

```
func mutate<Input>(  
  _ input: InputNode,  
  to newValue: Input.Value,  
  transaction: Transaction  
) {  
  input.value = newValue  
  
  for dependent in graph.dependents(of: input) {  
    dependent.markDirty()  
  }  
  
  scheduler.enqueue(transaction)  
}  
  
func commitPendingTransaction() {  
  for node in graph.dirtyNodesInDependencyOrder() {  
    node.recomputeIfNeeded()  
  }  
  commitLayoutAndPresentation()  
}
```

26. 经典 ObservableObject 在依赖图中的对象级失效

```
final class UserModel: ObservableObject {  
  @Published var name = "Tom"  
  @Published var age = 20  
}  
  
struct NameView: View {  
  @ObservedObject var model: UserModel  
  
  var body: some View {  
    Text(model.name)  
  }  
}
```

`@ObservedObject` 监听模型的 `objectWillChange`。任一 `@Published` 属性发出对象变化通知时，声明并观察该对象的 View 成为失效入口，即使该 View 的 body 没读取发生变化的字段。

对象级失效与最小渲染可以同时成立

model.age 改变
↓ objectWillChange
所有观察 model 的相关 View 节点失效
↓ 各自 body 重新求值
输出未改变的节点仍可能无需真实渲染更新

`@StateObject` 与 `@ObservedObject` 在经典通知粒度上相近：都响应 `objectWillChange`。两者的核心差别是所有权与生命周期，而不是字段级与对象级。

包装器	对象由谁创建/拥有	Identity 不变时	通知机制
StateObject	当前 View 身份	保持同一对象	objectWillChange

包装器	对象由谁创建/拥有	Identity 不变时	通知机制
ObservedObject	外部	取决于外部所有者	objectWillChange
EnvironmentObject	祖先注入方	随注入作用域	objectWillChange

27. Observation : 属性访问级追踪

```
import Observation

@Observable
final class UserModel {
    var name = "Tom"
    var age = 20
}

struct NameView: View {
    let model: UserModel

    var body: some View {
        Text(model.name)
    }
}
```

从 iOS 17 等系统开始，Observation 让 SwiftUI 根据 body 实际读取的 observable property 建立依赖。`NameView` 直接读取 `name`，因此 `name` 变化会使它失效；只改变未读取的 `age` 通常不会。

27.1 `@Observable` 宏的概念展开

教学伪代码 | Observable 属性的访问与 mutation 注册 推导依据：Observation 公开提供 ObservationRegistrar、access 与 withMutation；Xcode 的 Expand Macro 可观察当前工具链生成的访问器。目的：说明字段级依赖为什么来自 getter 的实际读取，而 mutation 又如何使相关读者失效。边界：宏的真实展开、访问器名称和 registrar 细节会随 Swift 版本变化；应通过 Xcode 的“Expand Macro”核对当前工具链。

教学伪代码（非 Apple 源码）

```
final class UserModel: Observable {
    private let registrar = ObservationRegistrar()
    private var _name = "Tom"

    var name: String {
        get {
            registrar.access(self, keyPath: \.name)
            return _name
        }
        set {
            registrar.withMutation(of: self, keyPath: \.name) {
                _name = newValue
            }
        }
    }
}
```

27.2 `withObservationTracking` 的意义

```
withObservationTracking {
    print(model.name) // 本轮追踪读取 name
} onChange: {
    print("本轮读取的某个属性将发生变化")
}
```

依赖集合来自闭包本次真正执行过的访问，而不是静态分析代码里出现过的所有属性。回调的具体时序、一次性追踪语义和并发注意事项应以当前 Observation 文档为准。

28. `@State + @Observable` 与 `@Bindable`

现代 Observation 体系把“对象引用的所有权”与“对象内部字段的可观察性”拆开：

```
@Observable
final class FormModel {
    var name = ""
    var email = ""
}

struct FormView: View {
    @State private var model = FormModel() // View 身份持有引用

    var body: some View {
        @Bindable var model = model // 生成属性 Binding
        TextField("Name", text: $model.name)
    }
}
```

工具	负责什么	不负责什么
@State	在 View 身份下保持 model 引用	不会让普通 class 的内部字段自动可观察
@Observable	为属性访问与 mutation 生成观察支持	不自动决定对象由哪个 View 持有
@Bindable	从 Observable 对象的可写属性生成 Binding	不拥有模型
@Binding	读写某一个源值	不存储源值

29. 失效边界：字段级追踪最终仍映射到 View 级 body

Observation 能把依赖细化到属性，但 SwiftUI 的更新入口仍然通常是某个 View 的 body 计算，而不是只执行 body 中的一行。

```
struct UserView: View {
    let model: UserModel

    var body: some View {
        VStack {
            Text(model.name)
            Text("\(model.age)")
        }
    }
}
```

这个 `UserView.body` 同时读取 name 和 age，任一变化都能使它重新求值。若想缩小失效边界，可以拆为 `NameView` 与 `AgeView`，并让属性访问发生在最接近实际消费位置的 View 中。

性能原则 把 View 拆小不是为了追求文件数量，而是为了建立有意义的依赖与计算边界。先用 Instruments 测量，再针对高频或昂贵 body 优化。

第七部分 从状态写入到屏幕更新

30. 完整更新流水线

下面这条链把前面的概念串起来。它是分阶段的解释模型：真实 SwiftUI 可能批处理、合并事务、延迟到下一帧，并在不同平台采用不同呈现后端。

从事件到呈现

1 用户事件 / 异步结果
2 State、ObservableObject 或 Observation 属性发生 mutation
3 相关依赖被标记失效，更新进入事务/调度
4 定位已知 Identity 的 View 节点
5 DynamicProperty.update() 准备动态输入
6 相关 body 重新求值，产生新的 View values
7 按 Identity 协调旧节点与新描述
8 必要时重新布局、动画并提交呈现变化

教学伪代码 | 一次更新事务的总体调度 推导依据：状态写入会使依赖计算失效；SwiftUI 会在后续更新周期准备 DynamicProperty、求值 body、协调身份并提交呈现。目的：把前六部分的局部机制串成一次可顺序追踪的教学模型，同时提醒读者框架可以批处理多次 mutation。边界：真实 SwiftUI 可合并、延迟、分层求值或跳过工作；这里不承诺循环顺序、线程或一帧内的提交次数。

教学伪代码（非 Apple 源码）

```
func commitUpdateTransaction() {
    let dirtyComputations = dependencyGraph.takeDirtyNodes()

    for computation in dirtyComputations {
        let node = identityTree.node(for: computation.ownerID)
        var viewValue = node.latestInputValue

        prepareDynamicProperties(
            in: &viewValue,
            context: node.context
        )
        let newDescription = evaluateBody(viewValue)
        reconcile(node, with: newDescription)
    }

    layoutIfNeeded()
    applyAnimationsAndCommitPresentation()
}
```

31. 一次 `count += 1` 的逐步拆解

```
struct CounterView: View {
    @State private var count = 0

    var body: some View {
        VStack {
            Text("Count: \(count)")
            Button("Add") { count += 1 }
        }
    }
}
```

1. 按钮动作执行。读取 count，然后通过 State 的 nonmutating setter 写入新值。
2. SwiftUI 管理的状态位置更新，并使读取该状态的相关计算失效。
3. 框架在合适的更新周期处理失效，而不是要求开发者手动调用刷新。
4. CounterView 当前逻辑节点的 Identity 已知；SwiftUI 准备其动态属性。
5. CounterView.body 重新求值，构造新的 VStack、Text 和 Button 描述值。
6. SwiftUI 用 Identity 将新描述与旧逻辑节点匹配：VStack、Text、Button 仍是原来的逻辑位置。
7. 差异显示 Text 内容从 `Count: 0` 变为 `Count: 1`；Button 描述未发生可见变化。
8. 若文字尺寸变化，相关布局节点重新计算；最终只提交必要的呈现更新。

32. 三个“更新”不能混为一谈

层次	含义	常见误解
父 body 重算	父 View 的描述函数再次执行	等于整棵子树所有 body 都执行
子节点继续求值	框架决定是否深入某个自定义子 View	只看构造了 Child() 就能断言 Child.body 次数
布局更新	尺寸提议、测量与放置受影响	文字变化必然全屏重新布局
真实呈现更新	提交必要的绘制/图层/平台视图变化	body 重算就删除重建 UIView

因此打印日志是有价值的，但日志只证明某个代码路径执行了。不能从 `Parent.body` 打印一次推导“整个屏幕重绘了一次”，也不能把 `onAppear` 当作 View struct init 的计数器。

33. 父 body 重算后，子 body 是否必定重算

父 body 重算会重新构造子 View value，例如再次执行 `StaticChild()` 初始化表达式。但 SwiftUI 是否继续求值 `StaticChild.body`，受到身份、输入、框架优化与动态依赖影响。业务正确性不能依赖某个版本恰好跳过或执行了几次。

```
struct Parent: View {
    @State private var count = 0

    var body: some View {
        VStack {
            Text("\(count)")
            StaticChild() // 会重新构造 value；
            // body 是否继续求值不应作为业务契约
        }
    }
}
```

可靠规则 `body` 必须便宜、确定且无副作用。把网络请求、对象创建、数据库写入等工作放到 body，是比“它到底会执行几次”更根本的错误。

34. SwiftUI 与 UIKit 的职责差异

UIKit / AppKit	SwiftUI
长期 class 对象表示控件	短生命周期 View value 描述界面
命令式修改属性	通过状态重新声明所需界面
对象地址提供自然身份	结构身份或显式身份
状态常保存在对象中	State/模型存储与 View value 分离
开发者直接协调大量更新	运行时依赖、协调、布局与提交

SwiftUI 最终仍然依赖平台的布局、文本、图形、Core Animation 与 GPU 能力，但“一个 `Text` 就等于一个 `UILabel`”不是稳定保证。SwiftUI 可以采用平台视图、绘制层或其他实现策略。

35. Diff / reconciliation 到底比较什么

“SwiftUI 做 diff”是方便的概括，但不要把它机械等同于 React Virtual DOM 的逐节点字符串比较。SwiftUI 拥有编译期泛型结构、Identity、动态属性和依赖图，可以在多个层次协调更新。

- Identity 决定新旧描述是否对应同一逻辑节点。
- 同一节点的输入或描述变化决定是否需要更新其内容。
- 依赖图决定哪些计算先成为失效入口。
- 布局系统根据尺寸提议与子 View 测量决定放置。
- 事务与动画影响变化如何随时间呈现。

教学伪代码 | 按 Identity 协调旧节点与新描述 推导依据：公开行为表明稳定 identity 会保留 State，而 identity 改变会结束旧节点生命周期；同一节点的输入变化可产生内容或布局更新。目的：区分“同一节点更新”和“旧节点销毁、新节点创建”，避免把 diff 误解为只比较文本或 struct 是否相等。边界：SwiftUI 的真实协调算法、相等性优化和节点层次未公开，也不能机械类比为 React Virtual DOM 算法。

教学伪代码（非 Apple 源码）

```
func reconcile(oldNode, newDescription) -> Node {
    let newID = computeIdentity(newDescription)

    guard oldNode.identity == newID else {
        tearDownStateAndSubscriptions(oldNode)
        return createNode(
            identity: newID,
            initialDescription: newDescription
        )
    }

    oldNode.updateInputs(from: newDescription)
    if oldNode.isDirty || inputsRequireEvaluation(oldNode) {
        let newChildren = evaluateIfNeeded(oldNode)
        reconcileChildrenByIdentity(
            oldNode.children,
            newChildren
        )
    }
    return oldNode
}
```

第八部分 状态工具的实现模型与选择

36. 状态工具总表

工具	存储/所有权	变化粒度	典型用途
@State	SwiftUI；当前 View 身份	具体状态值	本地 UI 状态、持有 @Observable 对象
@Binding	无；代理源状态	由源决定	子 View 读写父状态
@StateObject	SwiftUI；当前身份持有对象	对象级 objectWillChange	旧 ObservableObject 体系的对象所有者
@ObservedObject	外部持有	对象级 objectWillChange	观察外部 ObservableObject
@EnvironmentObject	祖先注入方	对象级 objectWillChange	跨层级共享旧式模型
@Observable	普通对象；所有权另行决定	属性访问级	现代模型
@Bindable	无；包装 Observable 引用	由属性依赖决定	为属性生成 Binding
@Environment	祖先环境	对应环境键	主题、Locale、依赖等上下文

37. 状态所有权的四问

- 谁创建它？
- 谁决定它何时销毁？
- 谁有权修改它？
- 哪些 View 需要观察它？

把状态放在所有消费者的最低公共拥有者处。太低会在页面/分支消失时丢失，太高会扩大依赖范围并让生命周期与业务语义错位。

38. 外部输入复制进 State：何时正确

语义	推荐做法
只读显示父输入	普通 let/var 属性
子 View 直接编辑父事实来源	@Binding
第一次取父值，之后成为独立草稿	@State(initialValue:); 明确冲突规则
业务模型由页面拥有	@State + @Observable，或旧系统的 @StateObject
模型由协调器/父级拥有	普通引用 + Observation，或 @ObservedObject

39. `@StateObject` 的“只创建一次”需要限定 Identity

更准确的说法是：同一个 View Identity 只在建立持久对象时求值 `@StateObject` 保存的创建闭包。以后即使父 body 再次构造同位置的新 View value，只要身份不变，SwiftUI 仍复用已有对象，新的创建闭包通常不会被求值；不是“先创建一个候选对象再丢弃”。若业务实体切换应该产生新对象，应重新审视所有权，或在确有必要时用稳定业务 ID 表达新身份。

谨慎使用 `.id()` 改变 `.id()` 会重置整个子树中与身份绑定的状态，不应当只为了“强制刷新”。它表达的是“这是另一个逻辑实体”。

40. ObservableObject 与 Observation 的性能差异

差异主要发生在“哪些 View 成为失效入口”，而不是进入失效后是否采用两套不同的渲染引擎。两者最终都经过 SwiftUI 的 body 求值、身份协调、布局与呈现流程。

更新入口的粒度



常见误区与纠正

误区	更准确的理解
State 自己就是外部存储指针	State 是公开 wrapper；外部位置是合理模型，具体布局不公开。
nonmutating 是因为省略 self	是否改变值语义 self 与是否写出 self 无关。
View init 表示屏幕对象新建	只表示新的 View value 被构造。
title 这个 let 变化会改变 identity	普通输入值变化改变 View value；identity 取决于结构/类型/显式 ID。
State 精准只刷新 Text	安全表述是依赖该 State 的 View 计算节点失效；内部粒度不作过度断言。
ObservedObject 让所有后代 body 必执行	观察者 View 是失效入口；后代是否继续求值由框架协调。
Binding 要保存父 View 的 identity	Binding 保存读写源值的能力；闭包/位置已指向正确来源，不负责重建身份查找。
StateObject 每轮先创建候选对象再丢弃	它保存创建表达式的 autoclosure；身份已有对象时通常不会求值该闭包。
DynamicProperty.update 修改 wrapper 会递归刷新 View	update 修改的是框架准备求值的 View 工作副本，不是一次新的用户状态 mutation。
Environment 是一个全局字典	它是沿 View 层级传播的值集合；modifier 只覆盖其后代作用域。
EnvironmentObject 的祖先一定会随对象变化重算	只有实际观察/读取对象的 View 成为观察入口；单纯注入不等于订阅。
body 重算等于真实 UI 重绘	body 只重新产生描述，后续仍有协调、布局、呈现。
`.id(UUID())` 是通用刷新按钮	它持续制造新身份并重置整个子树生命周期。
`.onAppear` 能证明 struct 只初始化一次	onAppear 描述节点进入呈现层级，不是 struct init 计数。

调试 SwiftUI 的实用方法

41. 先定位问题属于哪一层

现象	优先检查
@State 突然重置	Identity、条件分支、ForEach ID、`.id()`、导航移除
ViewModel 重复创建	所有权：StateObject/State 是否放对；是否在 body 创建对象
body 次数很多	失效源、观察范围、父节点输入、环境变化
body 重算但屏幕没变化	正常：描述相同，后续无可见提交
列表行状态串了	是否以 index、不稳定 hash 或临时 UUID 作为 ID
异步任务重复	`.task` 与 Identity、参数 ID、幂等保护
Observation 没刷新	属性是否真正被 body 读取；模型是否用了 @Observable
ObservableObject 刷新太广	是否把大对象传给过多 View；可否拆分或迁移 Observation
EnvironmentObject 运行时崩溃	注入是否位于消费 View 的祖先链；类型是否完全一致；Preview/Test 是否单独注入
Environment 值不是预期值	从消费节点向上检查最近一次同 key 覆盖，以及 sheet/navigation 等实际层级
Binding 写到了错误数据	检查 Binding 的创建来源、ForEach 的稳定 ID，以及是否捕获了错误元素/index

42. 日志探针：能证明什么，不能证明什么

```
struct BodyProbeView: View {
    let name: String

    var body: some View {
        let _ = print("BODY:", name)
        return Text(name)
        .onAppear { print("APPEAR:", name) }
        .onDisappear { print("DISAPPEAR:", name) }
    }
}
```

日志	能说明	不能说明
BODY	这个 body getter 被求值	底层所有像素都重绘
APPEAR	逻辑内容进入可呈现层级	View struct 只初始化一次
DISAPPEAR	内容离开可呈现层级	State 已在此刻同步释放
init/deinit	某个 Swift value 或 class 对象创建/释放	直接等价于 SwiftUI identity

43. 六组最小实验

43.1 State 保持

```
struct StateExperiment: View {
    @State private var count = 0
    var body: some View {
        let _ = print("StateExperiment.body")
        return Button("\(count)") { count += 1 }
    }
}
```

观察：body 多次执行，count 仍持续增长。结论：View value/body 生命周期与状态存储生命周期不同。

43.2 条件身份重置

```
VStack {
    Button("Toggle") { show.toggle() }
    if show {
        CounterView()
    }
}
```

把 CounterView 加到 5，隐藏后再显示。若回到 0，说明该分支身份离开树后，局部 State 生命周期结束。

43.3 ObservableObject 对象级失效

```
final class Model: ObservableObject {
    @Published var name = "Tom"
    @Published var age = 20
}

// body 只显示 name，再修改 age，观察 body 日志。
```

43.4 Observation 属性级依赖

```
@Observable final class Model {
    var name = "Tom"
    var age = 20
}

// NameView.body 只读取 name。
// 分别修改 age 与 name，比较 body 日志。
```

预期：只改 age 时，若 `NameView.body` 从未读取 age，它通常不会因为该字段而失效；改 name 时会。

43.5 Binding 不保存父 Identity

```
struct Parent: View {
    @State private var name = "Tom"
    var body: some View {
        Editor(name: $name)
    }
}

struct Editor: View {
    @Binding var name: String
    var body: some View {
        TextField("Name", text: $name)
    }
}
```

在 Editor 中修改文本，再让 Parent 因无关状态重算。只要父 State 的身份仍存在，新的 Binding 仍通过父本轮传入的读写通道访问同一个 State location；Binding 无需记住 Parent 的 identity。

43.6 Environment 的最近祖先覆盖

```
VStack {
    ThemeProbe()
    ThemeProbe()
    .environment(\.colorScheme, .dark)
}
```

两个 `ThemeProbe` 处于不同环境作用域。第二个从最近覆盖读取 dark，第一个继续继承外层值。这比“所有 View 共用一个全局字典”更接近真实语义。

44. Instruments 与测量原则

日志适合验证语义，性能问题应使用 Xcode 的 SwiftUI Instruments、Time Profiler、Animation Hitches 与 signpost 等工具。不要把 body 次数当作唯一性能指标：一次昂贵求值可能比一百次极轻量求值更重要。

- 先记录基线：操作步骤、设备、系统版本、数据规模。
- 区分更新原因、body 时长、布局成本与绘制/提交成本。
- 检查大列表中的不稳定 ID、昂贵格式化、同步 I/O 和图片解码。
- 缩小观察范围前先确认热点，避免为了“减少刷新”破坏清晰的数据流。

结语：用一个统一模型理解 SwiftUI

完整心智模型

View value：当前状态下的界面描述
Identity：跨更新识别逻辑节点
Lifetime：身份存在期间保存状态与运行时资源
DynamicProperty：body 前准备动态输入
Dependencies：记录计算读取了哪些输入
Invalidation：输入变化使相关计算过期
Reconciliation / Layout / Render：把新描述变成最小必要呈现变化

最终公式 $UI = f(State)$ 是入口，但不是全部。真正的 SwiftUI 模型还需要 Identity 决定“谁是谁”、Lifetime 决定“什么持续存在”、Dependencies 决定“何时重算”。

附录 A 六种状态包装器的代码展开对照

Property Wrapper 的语言展开只告诉我们 `\_property`、wrappedValue 和 projectedValue 的关系；它不会显示 SwiftUI 在运行时怎样为 DynamicProperty 安装状态位置、环境和订阅。下面把六种常用写法放在一起，作为阅读源码时的索引。

声明	包装器存储	wrapped value	projected value
@State var count	_count: State<Int>	count: Int, 可读写	\$count: Binding<Int>
@Binding var name	_name: Binding<String>	name: String, 可读写	\$name: Binding<String>
@ObservedObject var model	_model: ObservedObject<Model>	model: Model 引用	\$model: OO.Wrapper
@StateObject var model	_model: StateObject<Model>	model: Model 引用	\$model: OO.Wrapper
@Environment(\.key) var value	_value: Environment<Value>	value: Value	通常不使用投影
@EnvironmentObject var model	_model: EnvironmentObject<Model>	model: Model 引用	\$model: EO.Wrapper

A.1 State 与 Binding

```
// @State private var count = 0
private var _count = State<Int>(wrappedValue: 0)
var count: Int {
  get { _count.wrappedValue }
  nonmutating set { _count.wrappedValue = newValue }
}
// `$count` 访问 _count.projectedValue

// @Binding var name: String
private var _name: Binding<String>
var name: String {
  get { _name.wrappedValue }
  nonmutating set { _name.wrappedValue = newValue }
}
// `$name` 仍是 _name.projectedValue
```

A.2 ObservedObject 与 StateObject

```
// @ObservedObject var model: Model
private var _model: ObservedObject<Model>
var model: Model { _model.wrappedValue }
// `$model` 访问 _model.projectedValue

// @StateObject private var model = Model()
private var _model =
  StateObject<Model>(wrappedValue: Model())
var model: Model { _model.wrappedValue }
// Model() 被 autoclosure 包装; `$model`
// 访问 ObservedObject<Model>.Wrapper
```

A.3 Environment 与 EnvironmentObject

```
// @Environment(\.colorScheme) var scheme
private var _scheme =
  Environment<ColorScheme>(\.colorScheme)
var scheme: ColorScheme { _scheme.wrappedValue }

// @EnvironmentObject var session: SessionModel
private var _session =
  EnvironmentObject<SessionModel>()
var session: SessionModel { _session.wrappedValue }
// `$session` 访问 _session.projectedValue
```

展开之后仍缺少什么 这些代码没有表现 View Identity、State location、Environment propagation、objectWillChange subscription 和 invalidation。它们属于 SwiftUI 在 `\_makeProperty` / DynamicProperty 准备阶段安装的运行时关系，不能仅靠 Property Wrapper 语法推出。

附录 B 术语速查

术语	一句话解释
wrappedValue	Property Wrapper 对外暴露的实际值。
projectedValue	通过 <code>`\$property`</code> 访问的投影值，State 的投影是 Binding。
nonmutating set	setter 不要求值类型 self 可变。
DynamicProperty	可在 body 前由 SwiftUI 更新底层值的动态存储属性协议。
Binding	不拥有值；封装对某个来源值的读取与写入能力。
StateObject	把一个 ObservableObject 的所有权和生命周期绑定到当前 View Identity。
ObservedObject	观察由外部拥有并传入的 ObservableObject。
EnvironmentValues	沿 View 层级传播、可按 key 被后代作用域覆盖的值集合。
EnvironmentObject	从环境层级按类型查找并观察经典 ObservableObject。
Structural identity	来自 ViewBuilder 静态结构、位置与类型的隐式身份。
Explicit identity	来自业务 ID 或 <code>`.id()`</code> 的显式身份。
Dependency tracking	运行时记录某个计算读取了哪些动态输入。
Invalidation	将相关计算标记为过期，等待重新求值。
Observation	Swift 的属性访问级观察系统。
AttributeGraph	SwiftUI 使用的私有依赖计算基础设施；细节不公开。
Reconciliation	按身份协调新旧描述，决定复用、更新或替换。
教学伪代码	依据公开 API、SDK interface 与可重复行为构建的解释模型；不等于 Apple 源码。

附录 C 复习问题

1. 为什么 ``self.box.value = newValue`` 可以出现在 nonmutating setter 中，而 ``self.box = Box(...)`` 不行？
2. 普通输入值变化与 View Identity 变化有什么区别？
3. 为什么 State 的 initialValue 在同一 Identity 下可能被忽略？
4. 为什么 DynamicProperty.update 在 body 前能够工作？当前 View 的 Identity 从哪里来？
5. Binding 为什么不需要保存父 View 的 Identity？它如何访问正确的 State storage？
6. 为什么 State 的初始值是一个值，而 StateObject 的初始化表达式使用 autoclosure？
7. 在 iOS 17 Observation 中，为什么经常用 @State 持有 @Observable model？如何避免昂贵的重复初始化表达式？
8. EnvironmentValues 如何沿 View 树传播？多个相同 key 的值冲突时如何选择？
9. 为什么 @Environment.update() 的 mutating 行为不会形成新的状态写入循环？
10. EnvironmentObject 与 Environment 的共同点和额外能力分别是什么？
11. 祖先注入 EnvironmentObject 但从读取它时，对象变化会让祖先 body 重算吗？
12. ObservedObject 的 compiler expansion 能解释 ``$model.name``，但为什么不能单独解释 View invalidation？
13. 同一个 Child Identity 下，ObservedObject 从对象 A 切换到 B 时，哪些状态保留，哪条订阅需要替换？
14. StateObject 的 internal Storage 中 initially 与 object 两个 case 分别表达什么？
15. 为什么两个相邻的 `OwnedChild()` 会创建两个 StateObject，而父 body 重算不会让单个 OwnedChild 重建对象？
16. 对象级 invalidation 与最小渲染为什么不矛盾？
17. Observation 的“字段级”为什么仍然不是“只执行 body 中某一行”？

18. 为什么 `.id(UUID())` 通常是危险的刷新手段？

19. 一个 body 日志能证明什么，不能证明什么？

附录 D 资料与版本核对

以下资料用于核对公开 API 和概念边界。讲义中的私有实现图均为教学模型。

- 状态与动态属性：State、Binding、DynamicProperty、DynamicProperty.update()、Managing user interface state；入口均位于 developer.apple.com/documentation/swiftui/。
- 经典对象系统：ObservableObject、StateObject、StateObject.init(wrappedValue:)、Combine.ObservableObject.objectWillChange，以及 WWDC20 《Data Essentials in SwiftUI》。
- 环境与 Observation：Environment、EnvironmentValues、EnvironmentObject、Observation、Managing model data in your app、Migrating from ObservableObject to the Observable macro。
- Identity 与实现边界：WWDC21 《Demystify SwiftUI》；本机 Xcode 26.5 iPhoneSimulator SDK 的 SwiftUICore.swiftinterface 仅用于版本核对，不视为稳定公开 API。